ECS7032P - Statistical Planning and Reinforcement Learning

TAG: Tabletop Games Framework

Lab 4 - Advanced AI Agents: Evolutionary Agents and Sushi Go!

Before starting this lab, **pull** from the TableTop repository to update your code with the latest version of the framework. You can also use the **Pull** option in GitHub Desktop to update your code.

1 Evolutionary Agents for Decision Making

1.1 Random Mutation Hill Climber

A Random Mutation Hill Climber (RMHC) is implemented in TAG, in *players.rmhc.RMHCPlayer*. This agent implements a simple RMHC that mutates sequences of actions and evaluate states found at the end of these sequences. It does not use selection or crossover, as other evolutionary algorithms do. The main classes of this agent are:

- `RMHCPlayer`: it's the class that extends from Player.java and implements the agent API. This class is in charge of running the RMHC algorithm by using objects of the class Individual.
- `Individual`: implements an RMHC individual: a sequence of actions represented as an array, and the logic required to mutate the individual and perform an evaluation to determine its fitness.
- `RMHCParams`: a class for defining the parameters of RMHC. The two parameters that can be set for RMHC are `horizon` (individual length) and `discountFactor` (to reduce rewards from farther look-aheads).

You can define parameters for RMHC via the RMHCParams object when using the constructor, or via JSON as seen in the previous MCTS exercise. If no heuristic is specified, the agent will use the *default* the heuristic evaluation function of the game that is being played. It is also possible to set the allocated budget for each action decision (the most relevant ones are: BUDGET_FM_CALLS, BUDGET_ITERATIONS and BUDGET_TIME), in the same way as is done for MCTSParams.

An example in code is as follows:

```
RMHCParams params = new RMHCParams();
params.horizon = 15;
params.discountFactor = 0.99;
params.heuristic = AbstractGameState::getHeuristicScore;
params.budgetType = PlayerConstants.BUDGET_ITERATIONS;
params.budget = 100; // 100 iterations

AbstractPlayer rmhcPlayer = new RMHCPlayer(params);
```

Then, *json/players/rmhc.json* contains a possible configuration JSON file for this agent:

```
{
  "class" : "players.rmhc.RMHCParams",
  "horizon": 10,
  "discountFactor": 0.9,
  "budgetType" : "BUDGET_TIME",
  "budget" : 40,
  "breakMS" : 0,
  "heuristic" : {
    "class" : "players.heuristics.ScoreHeuristic"
  }
}
```

Exercise 1 Set up a RunGames tournament using JSON files, so the RMHC agent plays the game Connect 4 versus a RandomPlayer (i.e. (a tournament with only 2 players) for 100 times. As before, use the parameter "matchups" to set the number of repetitions. Then, Provide RMHC with a the following RMHC parameters and values:

- horizon: 5
- discountFactor: 1.0
- budgetType: BUDGET_FM_CALLS
- budget: 1000

Run the and observe the results you get in the log files.

Finally, execute the same tournament again but varying the individual length of RMHC to set it to 20. What do you observe in the results? Does the win rate of each agent change significantly when using different values?

The critical lines code of the *_getAction()* method in RMHCPlayer look as follows:

```
public AbstractAction _getAction(AbstractGameState gameState, List<AbstractAction>
    allActions) {
  /* Some initializations for budget count */

  // Creates a best individual
  bestIndividual = new Individual(/* params */);

  // Run evolution
  boolean keepIterating = true;
  while (keepIterating) {

    // Runs an iteration of RMHC and updates bestIndividual if needed
    runIteration(stateObs);

    /* Budget checks. keepIterating set to false if must stop */
  }

  // Return the first action of best individual found
  return bestIndividual.actions[0];
}
```

Each iteration of the algorithm is run in the runIteration method, which looks as follows (code for budget control has been removed here):

```
private void runIteration(AbstractGameState stateObs) {

  // Creates a new individual as a copy of the best one found so far.
  Individual newIndividual = new Individual(bestIndividual);

  // Mutates the new individual AND evaluates it.
  int sUpdated = newIndividual.mutate(getForwardModel(), getPlayerID());

  // Keep new individual if better than current
  if (newIndividual.value > bestIndividual.value)
    bestIndividual = newIndividual;
}
```

Exercise 2 Explore the 2 methods mentioned above and see if you can understand what is the code doing at each step. Do not hesitate to use the debugger to inspect variables and run it step by step if needed. Ask if anything is not clear.

Finally, we'll explore the Individual class, which contains data about the sequence of actions and (the most important function), `mutate`, which performs a mutation and evaluates the individual. In terms of data, Individual has some important class members:

```
  // Actions in individual. Size of the array is the individual length.
  AbstractAction[] actions;

  // An array of the game states for each action applied in the individual.
  AbstractGameState[] gameStates;

```

```
  // Fitness, actual individual length (<= actions.length), discount factor.
  double value;
  int length;
  double discountFactor;

  // Random generator for mutation operator and heuristic for function eval
  private Random gen;
  IStateHeuristic heuristic;
```

This data is used to keep information about the individual, and for the `mutate()` function, which looks as follows:

```
public int mutate(AbstractForwardModel fm, int playerID){ {

  if (length > 0) {

    // startIndex determines WHERE the mutation happens in the individual
    int startIndex = 0;
    if (length > 1) {
        startIndex = gen.nextInt(length - 1);
    }

    // Game state to start the rollout from
    AbstractGameState gs = gameStates[startIndex];

    // Perform rollout (evaluation) and return number of FM calls taken
    return rollout(gs, fm, startIndex, playerID);
  }

  return 0;
}
```

This finally leads us to the rollout() method, which does the heavy lifting for mutation and evaluation. This is a longer method, which is mainly divided in 3 blocks:

```
private int rollout(/* List of parameters */){ {

  for (int i = 0; i < startIndex; i++) {
    /* Block 1: evaluate up to mutation position (startIndex) */
  }

  for (int i = startIndex; i < actions.length; i++) {
    /* Block 2: mutate individual and compute score up to the end */
  }

  // Part 3: Assign final fitness and return budget used.
  this.value = delta;
  return new Pair<>(fmCalls, copyCalls);
}
```

Let's dive now on how mutation and evaluation are done in this implementation of RMHC[1].

1.1.1 Mutation

Mutation happens at a certain gene, as determined by the `mutate()` function and passed to the `rollout()` method (parameter `startIndex`). First, the gene (action) that mutates changes to another random action among the available ones at the corresponding game state. Secondly, due to the characteristics of Tabletop games, in which actions at a given point are highly dependent on the actions executed previously, **all actions** from the mutation position until the end of the individual are determined at random[2].

The following snippet shows this mutation in action, from the `startIndex` position:

```
for (int i = startIndex; i < actions.length; i++){

  // Make sure game is not over.
  if (gs.isNotTerminal()) {
```

[1]This concrete implementation differs from what was seen in the lecture this year.
[2]As in an MCTS rollout, reason for the name of this method in RMHC.

```
5
6       // Copy the game state and retrieve list of available actions
7       AbstractGameState gsCopy = gs.copy();
8       List<AbstractAction> curActions = fm.computeAvailableActions(gsCopy);
9
10      // Pick a new action at random among the available ones.
11      AbstractAction action = null;
12      if (curActions.size() > 0) {
13        action = curActions.get(gen.nextInt(curActions.size()));
14      }
15
16      // Advance game state with random action
17      fm.next(gsCopy, action);
18
19      // Prepare next game state and assign new action to individual
20      gameStates[i + 1] = gsCopy;
21      actions[i] = action;
22    }else{
23      break;
24    }
25  }
```

1.1.2 Computing the fitness

In this implementation of RMHC, the fitness for the individual is the discounted sum of the evaluation of all states found during the sequence of actions[3], so:

$$f(s_t) = h(s_{t+1}) + \gamma \times h(s_{t+2}) + \gamma^2 \times h(s_{t+3}) + \cdots = \sum_{k=0}^{\infty} \gamma^k \times h(s_{t+1=k})$$

The fitness ($f(s_t)$) is the sum of all the rewards found ($h(s_i)$), each multiplied by the discount factor (γ) at its depth (k)[4]. $h(s_i)$ is the state evaluation function of the state s_i. A simple evaluation (that does not include mutation) would look like this:

```
1 for (int i = 0; i < actions.length; i++) {
2   double score = heuristic.evaluateState(gameStates[i+1], playerID);
3   delta += Math.pow(discountFactor, i) * score;
4 }
```

A similar code block to the one above is included in the `rollout()` method of the Individual class. First, in "Block 1" (as declared in the snippet before Section 1.1.1 above), for the evaluation up to the mutation position, and then in "Block 2", this time integrated with the mutation code.

Exercise 3 Explore the Individual class and the 2 methods mentioned above (`mutate` and `rollout`) to see if you can understand what is the code doing at each step. As always, use the debugger if needed and ask if anything is not clear.

1.2 Rolling Horizon Evolutionary Algorithms

A Rolling Horizon Evolutionary Algorithm (RHEA) is implemented in TAG, in the package *players.rhea*. The main classes are:

- `RHEAPlayer`: it's the class that extends from Player.java and implements the agent API. This class is in charge of running the evolutionary algorithm, by using objects of the class RHEAIndividual, and implements the selection and crossover operators.
- `RHEAIndividual`: implements an RHEA individual: a sequence of actions represented as an array, and the logic required to mutate the individual and perform an evaluation to determine its fitness.
- `RHEAParams`: a class for defining the parameters of RHEA.
- `RHEAEnums`: defines the available types of selection and crossover operator.

This agent has many similarities to RMHC, with a few differences highlighted in the rest of this section.

[3] In Reinforcement Learning terms, this is known as the *Discounted Return*.
[4] This is again similar to Delayed Rewards in MCTS rollouts, as seen in the MCTS Lecture.

1.2.1 Extra Parameters

Beyond `horizon` and `discountFactor` (as seen in RMHC), RHEAParams has some extra parameters:

- `populationSize`: number of individuals in the RHEA population.
- `eliteCount`: number of individuals promoted by elitism.
- `childCount`: indicates how many new individuals are created each generation (only the best 'populationSize' inviduals will be kept in the end, so normally childCount + eliteCount $\geq$ populationSize).
- `selectionType`: type of selection operator, which values are RHEAEnums.SelectionType.RANK and RHEAEnums.SelectionType.TOURNAMENT.
- i (only for Tournament selection): number of individuals that take part in a selection tournament for one parent before crossover.
- `crossoverType`: type of crossover operator, with values RHEAEnums.CrossoverType.NONE (first parent becomes offspring), RHEAEnums.CrossoverType.UNIFORM, RHEAEnums.CrossoverType.ONE_POINT, RHEAEnums.CrossoverType.TWO_POINT.
- *shiftLeft*: whether the shift buffer enhancement is activated or not.
- *mutationCount*: indicates how many genes in the individual must mutate when executing this operator.
- `useMAST`: if the MAST enhancement is enabled (we suggest ignoring this and keeping it to *false* for now)[5].

RHEAParams operates in the same way as RMHCParams. For instance, in code:

```
RHEAParams params = new RHEAParams();
params.horizon = 20;
params.populationSize = 10;
params.crossoverType = RHEAEnums.CrossoverType.UNIFORM;
params.heuristic = new CantStopHeuristic();
AbstractPlayer player = new RHEAPlayer(params);
```

And in JSON (see json/players/rhea.json):

```
{
  "class" : "players.rhea.RHEAParams",
  "horizon": 10,
  "discountFactor": 0.900,
  "populationSize": 10,
  "childCount": 10,
  "crossoverType": "UNIFORM",
  "eliteCount": 1,
  "selectionType" : "TOURNAMENT",
  "tournamentSize" : 5,
  "budgetType" : "BUDGET_TIME",
  "budget" : 40,
  "breakMS" : 0
}
```

Exercise 4 Set up a RunGames tournament between RHEAPlayer and RMHC using a JSON configuration file, so that they play the game **CantStop** for 100 times (use the parameter "matchups" to set the number of repetitions).
Then, set the following parameters for the RHEA agents:

- horizon: 10
- budgetType: BUDGET_FM_CALLS
- budget: 1000
- selectionType: TOURNAMENT
- tournamentSize: 3
- crossoverType: UNIFORM
- shiftLeft: true
- populationSize: 5

[5]MAST (Move-Average Sampling Technique) is a back-up variant that keeps a $Q(a)$ value of an action a independent of the state in which it was applied (similar to AMAF/RAVE in MCTS). For more information, see *H. Finnsson and Y. Bjornsson, "Simulation-Based Approach' to General Game Playing" in Proc. Assoc. Adv. Artif. Intell., Chicago, Illinois, 2008, pp. 259–264.*

- eliteCount: 1
- childCount: 4
- Make sure the **lengths** of the individuals of both algorithms (RHEA and RMHC) and their **budgets** are the same, so the comparison is fair.

Run it and observe the results you get in the output files of the tournament.
Finally, execute the same tournament again but varying some of the parameters of the RHEA algorithm (e.g the population size, selection procedure, elite count, etc.). Just make sure to keep always the same budget and search horizon between both RHEA and RMHC. What do you observe in the results? Does the win rate of each agent change significantly when using different values for the parameters of RHEA?

1.2.2 Selection Operator

The selection operator that RHEA must use is indicated in RHEAParams. There are two selection operators implemented, a rank and a tournament system, which can be found in the respective methods in RHEAPlayer called `tournamentSelection()` and `rankSelection()`. Here's the tournament selection method, which should hopefully be self-explanatory:

```
RHEAIndividual tournamentSelection() {
  RHEAIndividual best = null;

  // Pick tournamentSize individuals
  for (int i = 0; i < params.tournamentSize; ++i) {
    int rand = randomGenerator.nextInt(population.size());

    // Keep the best one
    RHEAIndividual current = population.get(rand);
    if (best == null || current.value > best.value)
      best = current;
  }
  return best;
}
```

1.2.3 Crossover Operator

The crossover operator that RHEA must use is also indicated in RHEAParams. There are three main crossover operators implemented: uniform, one-point and two-point crossover, which can be found in the respective methods in RHEAPlayer called `uniformCrossover()`, `onePointCrossover()` and `twoPointCrossover()`. Here's the uniform crossover method, which again should be easy to understand:

```
private RHEAIndividual uniformCrossover(RHEAIndividual p1, RHEAIndividual p2) {
  RHEAIndividual child = new RHEAIndividual(p1);
  int min = Math.min(p1.length, p2.length);
  for (int i = 0; i < min; ++i) {
    // Copy genetic material from one parent or the other at random
    if (randomGenerator.nextFloat() >= 0.5f) {
      child.actions[i] = p2.actions[i];
      child.gameStates[i] = p2.gameStates[i];
    }
  }
  return child;
}
```

1.2.4 A full RHEA iteration

The function `runIteration()` in RHEAPlayer.java puts all the pieces above together. Here's the summarised code snippet for this function:

```
private void runIteration() {
  List<RHEAIndividual> newPopulation = new ArrayList<>();

  for (int i = 0; i < params.childCount; ++i) {
    /* copy params.eliteCount elites from 'population' to 'newPopulation'*/
  }
```

```
7
8   for (int i = 0; i < params.childCount; ++i) {
9     /* Perform selection and crossover to generate params.childCount
10       new individuals and add them to 'population'*/
11  }
12
13  for (RHEAIndividual individual : population) {
14    /* All individuals of the (old + child) population are mutated */
15  }
16
17  // Sort the population according to the fitness of its individuals */
18  population.sort(Comparator.naturalOrder());
19
20  // The best ones are moved to 'newPopulation'
21  int howMany = Math.min(population.size(),
22                         params.populationSize - params.eliteCount);
23  for (int i = 0; i < howMany; ++i) {
24    newPopulation.add(population.get(i));
25  }
26
27  // newPopulation becomes the population for the next iteration.
28  population = newPopulation;
29
30  // Sort the population again (accounts for promoted elite individuals).
31  population.sort(Comparator.naturalOrder());
32
33 }
```

Exercise 5 Explore the runIteration() method in RHEAPlayer in detail to check your understanding of the code. As usual, make use of the debugger if needed and ask if anything is not clear.

2 Sushi Go! (Assignment 1 game)

Sushi Go! is a simple card game where players try to play combinations of cards that score them points.



Although the rules of the game are simple, this game incorporates certain aspects that make it interesting for AI research:

- The game is **stochastic**, as the deck of cards is shuffled three times (one for each round in the game) before cards are dealt to players.
- Each player has a set of hidden cards in front of them, which adds **partial observability** to the game (you can't observe the full true state of the game before decision making.
- There's a drafting mechanic, where hands are moved between players. This means that:
  1. You can address the partial observability aspect after a few moves, as you know the cards you are passing to your opponent. TAG's implementation of Sushi Go! does this for you: the cards of the opponents that you know they have will be assigned in the copy of the state agents receive.
  2. You have the chance of working on the implementation of an **opponent modelling** to also influence your actions.
- Finally, the information of the state becomes richer as the game progresses, which can inform the decisions of your agent.

In TAG you can run SushiGo as usual, using the Game and RunGames classes. When running with graphics on, you'll see the following interface, showing the state of the game (in this case, for 4 players:



Exercise 6 Learn about Sushi Go! and its rules. You can use the following resources:

- Sushi Go! rules: `https://www.fgbradleys.com/rules/rules2/SushiGo-rules.pdf`
- Sushi Go! How To Play video: `https://www.youtube.com/watch?v=uSN0NoaeNbk`

Once you've learned about the rules of the game, do the following:

- Play a few games yourself against AI(s) built in TAG (run it from Game.java with visuals).
- Set up a Sushi Go! round robin tournament (game name: 'SushiGo', use RunGames.java and JSON configuration files) between four players: RHEA, BasicMCTS, OLSA and Random. Set it so it plays a 100 games and observe the results in the tournament output files.

For the assignment, you are recommended to write your own Sushi Go! heuristic / state evaluation function for your agent. In this heuristic you will likely need to access elements of the game state. Methods that grant access to this information are in `games.sushigo.SGGameState`, and here is a summary of the main functions:

- **double getGameScore(int playerId)**: current game score for the player with ID 'playerId' (not counting points in the current round).
- **getPointsPerCardType(int playerId)**: points in the game so far for player with ID 'playerId'. This is only updated at the end of each full round. This returns an array of `Map<SGCardType, Counter>`. So to get the points scored so far with Maki rolls by player 2, the full code would be `int pointsInMakiRolls = state.getPointsPerCardType()[2].get(SGCardType.Maki)`. A full list of the supported card types (SGCardType is an Enum) is in the SGCard class.
- **Deck <SGCard> getPlayerHands()[playerId]**: returns the deck of cards that the player with ID 'playerId' has in their hand at this point. For opponent players with unknown hands, cards will be randomized to prevent disclosing the true state of the game when the agent's getAction() method is called.
- **boolean hasSeenHand(int playerId, int opponentId)**: indicates if the player with ID 'playerId' has seen the hand of the player with ID 'opponentId' previously this round. If this is `true` then the cards returned by the previous method can be relied on as accurate.
- **List<Deck<SGCard>> getPlayedCards()**: returns a list of the cards currently in front of each other player. This is a List indexed by the player number.
- **Counter getPlayedCardTypes(SGCardType type, int player)**: This is a helper function that is more intuitive than the previous method. It returns a Counter (a TAG Component) that holds the number of cards of the specified type played by the player over the current round so far - counter.getValue() gets the actual number as an int.
- **Counter getPlayedCardTypesAllGame(SGCardType type, int player)**: This is the same as the previous method, but includes all cards played in all previous rounds too.

Exercise 7 Implement a simple heuristic for SushiGo that achieves better results than the one provided by the `_getHeuristicScore` function described above. To test this, play 2-player games between identical AIs (ideally an advanced one - BasicMCTS or RHEA) with the only difference that one of them uses the default heuristic while the one you create is used by the other. Run tournaments aiming to achieve a consistent ($> 65\%$) victory rate over the default heuristic.

Finally, check if your heuristic also performs well when running 4 player games (setting the *nPlayers* parameter to 4 in the tourrnament configuration file).

After the lab: 1 Sushi Go! was implemented by some colleagues of the Game AI group from the University of Malmö, Sweden. After this, they published a (short) paper in 2021 where they show some results of running experiments in the game [1].

Read this 4-page paper to know more about what they did: `https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9618987`

The initial implementation of Sushi Go!, as detailed in the paper, used a perfect information version. This means that players could 'see' all the cards. This was corrected after the paper was published, and the version now in TAG is correctly imperfect information. This means that when an `AbstractPlayer` receives a copy of the `SushiGoGameState` all the cards that they have not seen, whether in other players' hands or in the draw deck, are shuffled. This prevents a player being able to make a decision using information they do not have, and we'll cover this in more detail in Lecture 5. For the moment, you should be aware that `BasicMCTSPlayer` does not take account of imperfect information at all, and assumes that the shuffled state it receives when making a decision is perfectly accurate.

References

[1] Carl-Magnus Embring Klang, Victor Enhörning, Alberto Alvarez, and Jose Font. Assessing simultaneous action selection and complete information in tag with sushi go! In *2021 IEEE Conference on Games (CoG)*, pages 01–04. IEEE, 2021.